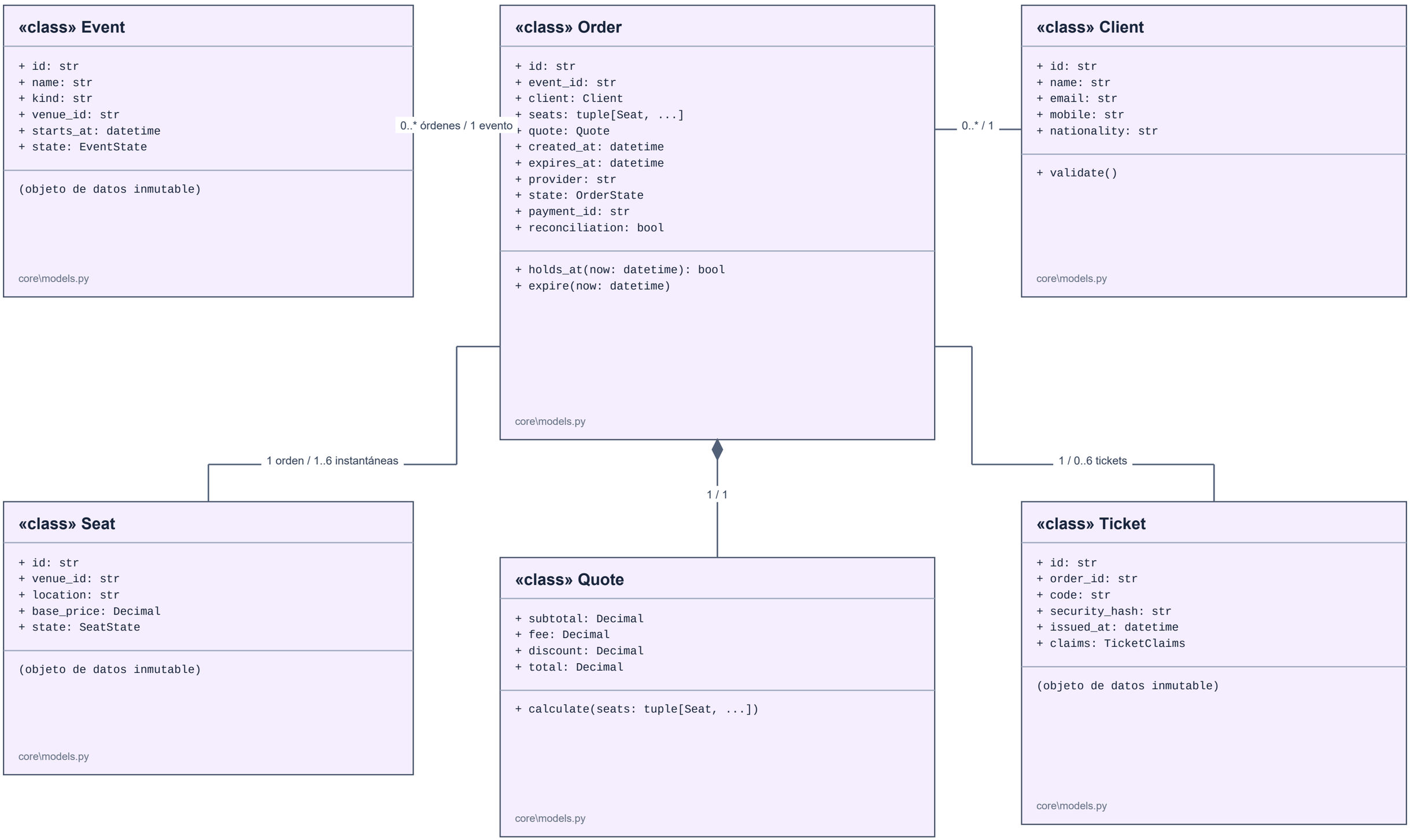
01 / Modelo de dominio

Entidades puras · nombres coincidentes con el código · atributos inmutables sin ORM



EventState: Programado | Agotado | Cancelado. SeatState: Disponible | Bloqueado | Vendido.
OrderState: Pendiente de Pago | Aprobada | Rechazada | Expirada. Fecha/hora UTC; dinero Decimal. TicketClaims: vista 09.
Línea = asociación; rombo lleno = composición. Los identificadores representan referencias entre agregados.

02 / Casos de uso y adaptadores primarios

El servicio implementa el contrato de entrada; HTTP y tareas programadas dependen del contrato

«interface» BookingUseCases

(sin atributos públicos)

```
+ events(): tuple[Event, ...]
+ availability(event_id: str): tuple[Seat, ...]
+ quote(event_id: str, seat_ids: tuple[str, ...]): Quote
+ reserve(event_id: str, client: Client, seat_ids: tuple[str, ...], provider: str):
  Order
+ confirm(provider: str, reference: PaymentReference): Order
+ expire(): int
+ get_order(order_id: str): Order
+ tickets(order_id: str): tuple[Ticket, ...]
+ ticket_svg(order_id: str, ticket_id: str): str
+ verify_ticket(order_id: str, ticket_id: str): bool
+ dispatch(): int
```

core\ports.py

implementa

«class» BookingService

```
- catalog: Catalog
- store: OrderStore
- clock: Clock; ids: IdSource
- gateways: dict[str, PaymentGateway]
- signer: TicketSigner; renderer: TicketRenderer
- notifications: dict[str, Notification]
- invoice: Invoice
```

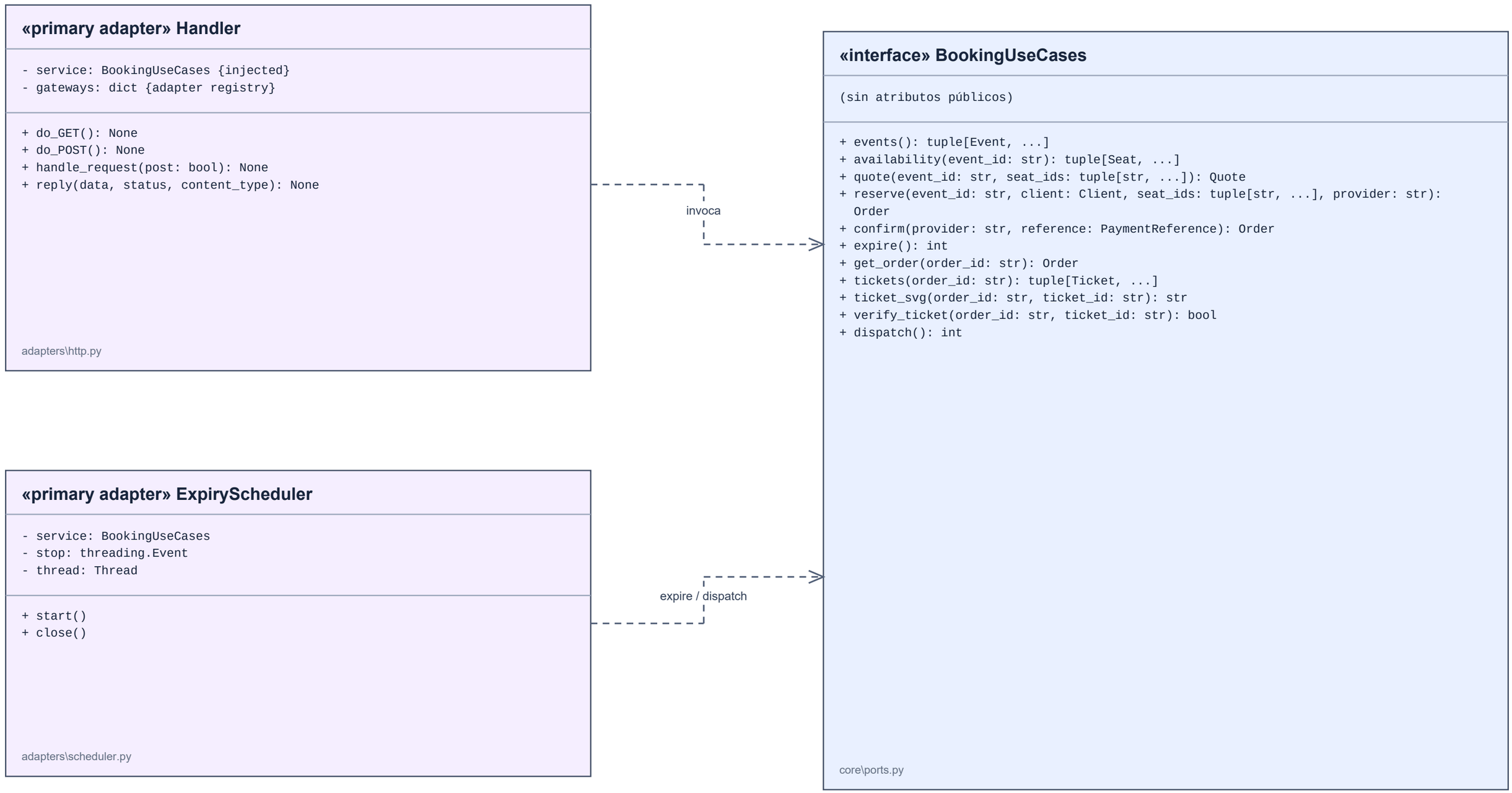
```
+ events(): tuple[Event, ...]
+ availability(event_id: str): tuple[Seat, ...]
+ quote(event_id: str, seat_ids: tuple[str, ...]): Quote
+ reserve(event_id: str, client: Client, seat_ids: tuple[str, ...], provider: str):
  Order
+ confirm(provider: str, reference: PaymentReference): Order
+ expire(): int
+ get_order(order_id: str): Order
+ tickets(order_id: str): tuple[Ticket, ...]
+ ticket_svg(order_id: str, ticket_id: str): str
+ verify_ticket(order_id: str, ticket_id: str): bool
+ dispatch(): int
```

core\service.py

Las dependencias de BookingService son contratos del núcleo; nunca importa adaptadores.
Línea discontinua con triángulo vacío = realización de interfaz. Se muestran los métodos públicos del contrato.
El ensamblaje de dependencias se realiza en bootstrap.build(). Todos los puertos están en core/ports.py.

03 / Entrada web y tareas temporales

Los adaptadores primarios traducen eventos externos a operaciones del núcleo



Handler es una clase interna de make_server(); recibe el servicio por cierre. También expone el endpoint de webhook.
El parser de cada pasarela transforma su notificación técnica a PaymentReference antes de invocar confirm().
Línea discontinua con flecha abierta = dependencia. No hay cálculo de tarifas en los adaptadores primarios.

04 / Catálogo y persistencia

Puertos definidos en el núcleo; implementaciones simuladas reemplazables

«interface» Catalog

(sin atributos públicos)

```
+ events(): tuple[Event, ...]
+ event(event_id: str): Event
+ seats(venue_id: str): tuple[Seat, ...]
```

core\ports.py

implementa

«mock adapter» MySqlCatalogMock

```
- data: tuple[Event, ...]
```

```
+ events(): tuple[Event, ...]
+ event(event_id: str): Event
+ seats(venue_id: str): tuple[Seat, ...]
```

adapters\storage.py

«interface» OrderStore

(sin atributos públicos)

```
+ transaction(): ContextManager[None]
+ orders(): tuple[Order, ...]
+ order(order_id: str): Order
+ save_order(order: Order): None
+ tickets(order_id: str): tuple[Ticket, ...]
+ save_ticket(ticket: Ticket): None
+ deliveries(): tuple[Delivery, ...]
+ save_delivery(delivery: Delivery): None
```

core\ports.py

implementa

«mock adapter» MongoOrderStoreMock

```
- _orders: dict[str, Order]
- _tickets: dict[str, Ticket]
- _deliveries: dict[str, Delivery]
- _lock: RLock
```

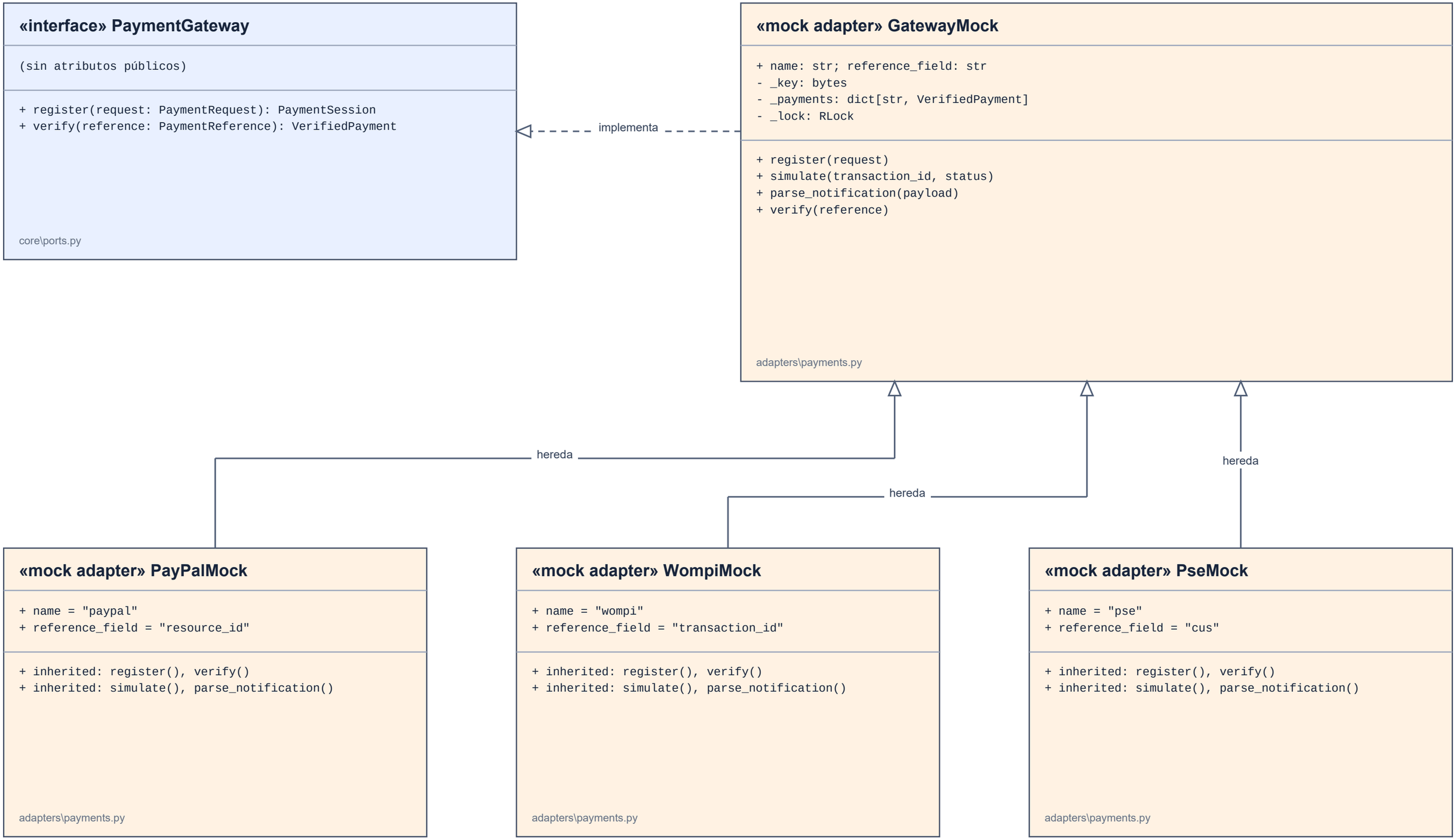
```
+ transaction(): ContextManager[None]
+ orders(): tuple[Order, ...]
+ order(order_id: str): Order
+ save_order(order: Order): None
+ tickets(order_id: str): tuple[Ticket, ...]
+ save_ticket(ticket: Ticket): None
+ deliveries(): tuple[Delivery, ...]
+ save_delivery(delivery: Delivery): None
```

adapters\storage.py

Destino arquitectónico: MySQL para catálogo legado; MongoDB para órdenes, tickets y bandeja de salida.
Esta entrega usa memoria: NO requiere ni conecta servidores de bases de datos. OrderStore garantiza atomicidad en un proceso.
RLock y snapshots pertenecen al adaptador. El núcleo no conoce conexiones, consultas SQL, documentos BSON ni bloqueos técnicos.

05 / Pagos externos

Contrato común; tres proveedores concretos con traducción técnica propia



Los payloads resource_id / transaction_id / cus son ficticios: no representan las API oficiales.
register() recibe PaymentRequest y devuelve PaymentSession; verify() recibe PaymentReference y devuelve VerifiedPayment.
El estado aprobado se verifica contra una transacción registrada y firmada; el núcleo valida importe, moneda, orden y vigencia.

06 / Tickets gráficos y criptografía

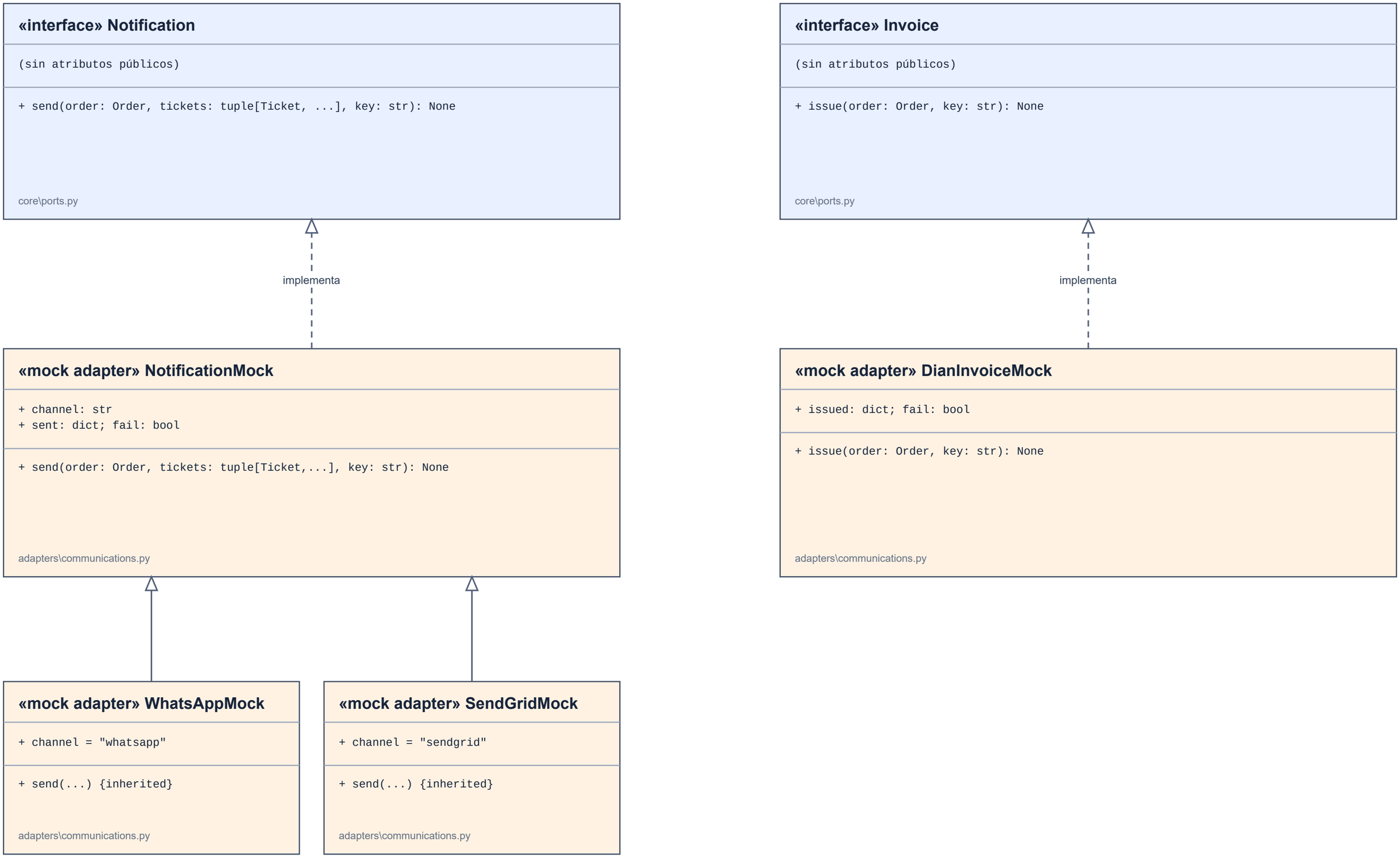
Dos adaptadores secundarios completamente implementados



HmacTicketSigner canoniza TicketClaims en JSON y firma con el secreto del evento; compare_digest verifica la firma. SvgTicketRenderer produce un SVG completo, legible y descargable con código alfanumérico, asiento y hash. No requiere QR. TicketClaims se detalla en la vista 09. Los algoritmos y formatos técnicos permanecen fuera del núcleo.

07 / Notificación y facturación

Contratos separados para mensajes y emisión de factura; salidas simuladas e idempotentes



BookingService crea Delivery por canal tras aprobar la orden. dispatch() reintenta pendientes sin emitir tickets nuevos.
Las clases mock registran resultados en memoria: no envían mensajes ni generan facturas electrónicas reales ante DIAN.

08 / Valores de pago y tiempo

Tipos del dominio intercambiados por los puertos; ninguna dependencia de SDK externos

«value object» PaymentRequest

```
+ order_id: str
+ amount: Decimal
+ currency: str
```

(objeto de datos inmutable)

core\models.py

«value object» PaymentSession

```
+ id: str
+ order_id: str
+ provider: str
```

(objeto de datos inmutable)

core\models.py

«value object» PaymentReference

```
+ transaction_id: str
+ proof: str
```

(objeto de datos inmutable)

core\models.py

«value object» VerifiedPayment

```
+ transaction_id: str
+ order_id: str
+ provider: str
+ amount: Decimal
+ currency: str
+ status: str
```

(objeto de datos inmutable)

core\models.py

«interface» Clock

(sin atributos públicos)

```
+ now(): datetime
```

core\ports.py

implementa

«adapter» SystemClock

(sin atributos públicos)

```
+ now(): datetime
```

bootstrap.py

«interface» IdSource

(sin atributos públicos)

```
+ new(): str
```

core\ports.py

implementa

«adapter» SecureIds

(sin atributos públicos)

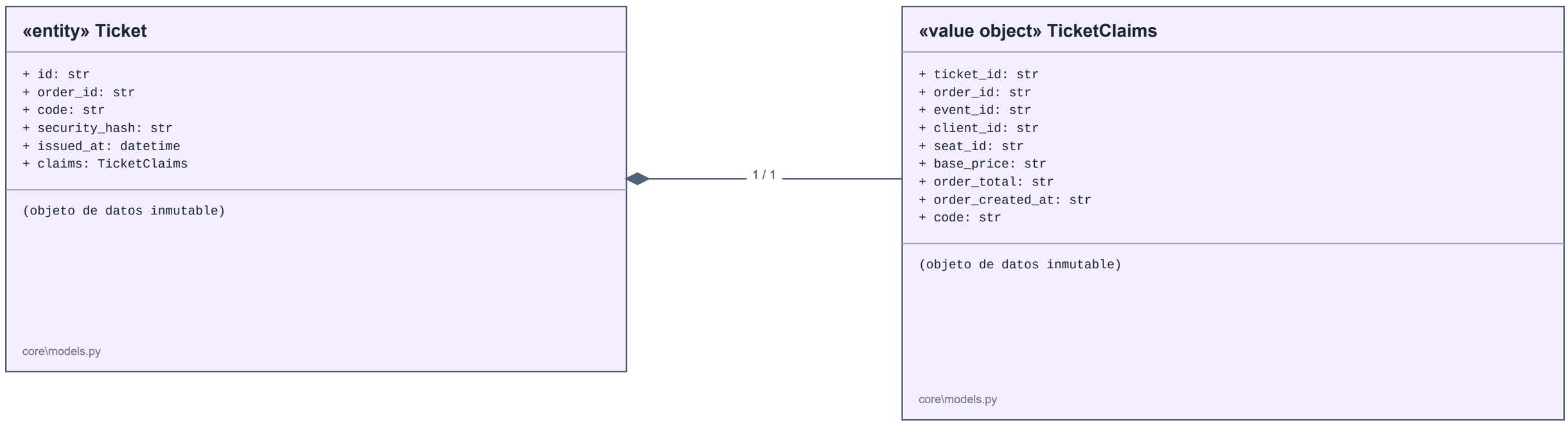
```
+ new(): str
```

bootstrap.py

Moneda de esta solución: COP. PaymentReference contiene una referencia y prueba opacas para el dominio, no un HTTPRequest. SystemClock usa UTC; SecureIds usa 128 bits aleatorios. En pruebas se inyecta ManualClock sin esperar diez minutos reales.

09 / Evidencia inmutable y entrega de efectos

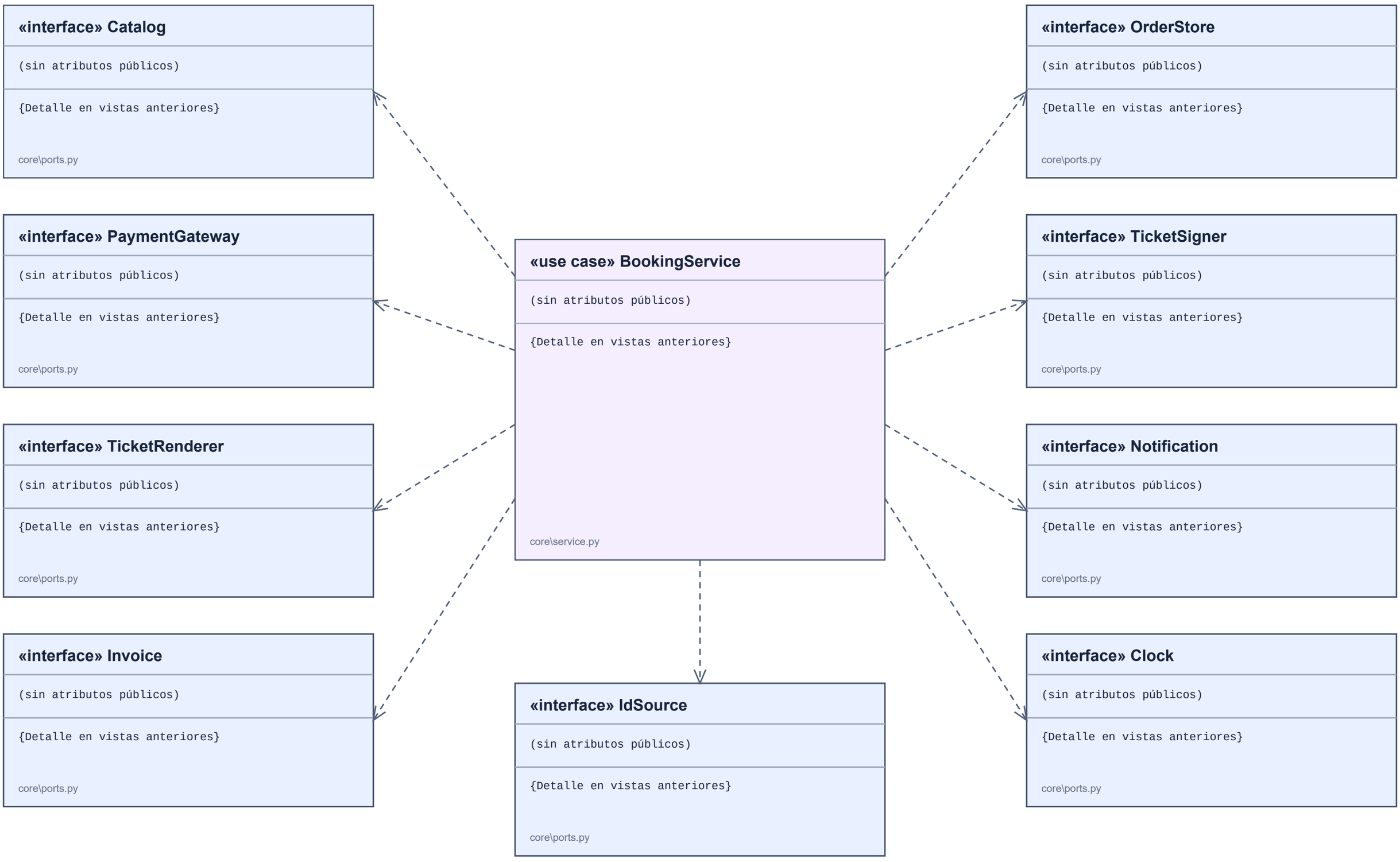
TicketClaims vincula la firma a la compra; Delivery permite reintentar salidas



Los claims incluyen ticket, orden, evento, cliente, asiento, precio, total, fecha de creación y código de canje.
La firma usa un secreto específico del evento. Delivery.id = order.id + canal: clave idempotente para efectos secundarios.
Los objetos son frozen dataclasses: las transiciones crean versiones nuevas mediante replace(), sin mutación compartida.

10 / Dependencias hacia puertos secundarios

Vista de conjunto: las cajas remiten a las clases detalladas en las páginas anteriores



En esta vista se omiten atributos y operaciones para destacar la dirección de las dependencias.
BookingService usa exclusivamente estos contratos. La composición concreta está fuera del núcleo, en bootstrap.py.
Las realizaciones de los adaptadores se muestran en las vistas 04 a 08; el contrato primario, en la vista 02.